

Documentation Complète - Projet Sécurisation d'un serveur WordPress

Projet TP – Déploiement et sécurisation d'un serveur WordPress

IP : 172.16.114.136

Hostname : Tkana--b1-cyber

PARTIE 1 - Section 1 : Préparation du système Debian

1.1 Mise à jour du système

```
apt update && apt upgrade -y
```

1.2 Installation des paquets de base

```
apt install sudo curl wget vim nano net-tools openssh-server -y
```

1.3 Création des utilisateurs

Utilisateur "youenn"

```
# Création de l'utilisateur
adduser Youenn

# Configuration :
# - Mot de passe : rtyfghvbn
# - Full Name : Youenn Duval
# - Autres champs : []
# - Confirmation : Y
```

Utilisateur "thanina" (étudiant)

```
# Création de l'utilisateur
adduser thanina

# Configuration :
# - Mot de passe : rtyfghvbn
```

```
# - Full Name : Thanina Kana
# - Autres champs : []
# - Confirmation : Y
```

1.4 Configuration des droits sudo

```
usermod -aG sudo youenn

usermod -aG sudo thanina
```

1.5 Vérification

```
# Vérifier l'appartenance au groupe sudo
groups youenn
groups thanina

# Tester la connexion
su - thanina
sudo whoami
# Résultat attendu : root
exit
```

PARTIE 1 - Section 2 : Installation du serveur web Apache

2.1 Installation d'Apache

```
apt install apache2 -y
```

2.2 Vérification du service

```
# Vérifier le statut
systemctl status apache2

# Activer au démarrage
systemctl enable apache2
```

2.3 Activation des modules nécessaires

```
# Module de réécriture d'URL
a2enmod rewrite

# Module pour les en-têtes HTTP
a2enmod headers

# Module SSL pour HTTPS
a2enmod ssl

# Redémarrer Apache
systemctl restart apache2
```

2.4 Création du répertoire DocumentRoot

```
# Créer le répertoire pour WordPress
mkdir -p /var/www/wordpress
```

2.5 Création du VirtualHost WordPress

```
# Créer le fichier de configuration
nano /etc/apache2/sites-available/wordpress.conf
```

Contenu du fichier `wordpress.conf` :

```
<VirtualHost *:80>
    ServerAdmin admin@wordpress.local
    ServerName wordpress.local
    DocumentRoot /var/www/wordpress

    <Directory /var/www/wordpress>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/wordpress_error.log
    CustomLog ${APACHE_LOG_DIR}/wordpress_access.log combined
</VirtualHost>
```

2.6 Activation du VirtualHost

```
# Activer le site WordPress
a2ensite wordpress.conf

# Désactiver le site par défaut
a2dissite 000-default.conf

# Tester la configuration
apache2ctl configtest
# Résultat attendu : Syntax OK

# Recharger Apache
systemctl reload apache2
```

2.7 Vérification

```
# Vérifier l'activation du VirtualHost
ls -l /etc/apache2/sites-enabled/

# Vérifier le DocumentRoot
ls -ld /var/www/wordpress
```

Configuration réseau

Nom de domaine local : `wordpress.local`

IP du conteneur : `172.16.114.136`

PARTIE 1 - Section 3 : Installation de MariaDB

3.1 Installation de MariaDB

```
apt install mariadb-server mariadb-client -y
```

3.2 Vérification du service

```
# Vérifier le statut
systemctl status mariadb

# Activer au démarrage
systemctl enable mariadb
```

3.3 Sécurisation initiale

```
mysql_secure_installation
```

Réponses fournies :

Question	Réponse	Justification
Enter current password for root	[]	Pas de mot de passe initial
Switch to unix_socket authentication?	n	Utilisation mot de passe classique
Change the root password?	Y	Sécurisation obligatoire
New password	rtyfghvbn	Accès administrateur MySQL
Remove anonymous users?	Y	Suppression comptes anonymes
Disallow root login remotely?	Y	Root uniquement en local
Remove test database?	Y	Suppression base de test
Reload privilege tables?	Y	Application des modifications

3.4 Création de la base de données WordPress

```
# Se connecter à MySQL
mysql -u root -p
```

```
-- Créer la base avec encodage UTF-8
CREATE DATABASE wordpress_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

3.5 Création de l'utilisateur dédié

```
-- Créer l'utilisateur WordPress
CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'WordPress2026!Secure';
```

3.6 Attribution des droits

```
-- Donner les droits uniquement sur wordpress_db
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';

-- Appliquer les modifications
FLUSH PRIVILEGES;
```

3.7 Vérification de la configuration

```
-- Vérifier les utilisateurs
SELECT user, host FROM mysql.user;

-- Vérifier les bases de données
SHOW DATABASES;

-- Vérifier les droits de wp_user
SHOW GRANTS FOR 'wp_user'@'localhost';

-- Quitter MySQL
EXIT;
```

Informations de connexion MySQL

Paramètre	Valeur
Nom de la base	wordpress_db
Utilisateur	wp_user
Mot de passe	WordPress2026!Secure
Hôte	localhost
Encodage	utf8mb4

PARTIE 1 - Section 4 : Installation de PHP

4.1 Installation de PHP et extensions

```
apt install php libapache2-mod-php php-mysql php-gd php-curl php-xml php-  
mbstring php-zip php-intl -y
```

4.2 Vérification de l'installation

```
# Vérifier la version de PHP
php -v
# Résultat : PHP 8.2.29

# Vérifier le module Apache
```

```
apache2ctl -M | grep php
# Résultat : php_module (shared)
```

4.3 Redémarrage d'Apache

```
systemctl restart apache2
```

4.4 Vérification finale

```
# Statut Apache
systemctl status apache2

# Vérifier que PHP est chargé
apache2ctl -M | grep php
```

PARTIE 1 - Section 5 : Installation de WordPress

5.1 Téléchargement de WordPress

```
# Aller dans le répertoire temporaire
cd /tmp

# Télécharger la dernière version de WordPress
wget https://wordpress.org/latest.tar.gz

# Extraire l'archive
tar -xzf latest.tar.gz

# Vérifier l'extraction
ls -l wordpress/
```

5.2 Déploiement dans le répertoire web

```
# Copier tous les fichiers WordPress
cp -r /tmp/wordpress/* /var/www/wordpress/

# Vérifier la copie
ls -l /var/www/wordpress/
```

Fichiers déployés :

- `wp-admin/` : Interface d'administration
- `wp-content/` : Thèmes, plugins, médias
- `wp-includes/` : Fichiers core WordPress
- `wp-config-sample.php` : Fichier de configuration exemple
- `index.php` : Point d'entrée

5.3 Configuration du fichier wp-config.php

```
# Aller dans le répertoire WordPress
cd /var/www/wordpress

# Copier le fichier exemple
cp wp-config-sample.php wp-config.php

# Éditer le fichier
nano wp-config.php
```

Configuration de la base de données :

```
define( 'DB_NAME', 'wordpress_db' );
define( 'DB_USER', 'wp_user' );
define( 'DB_PASSWORD', 'WordPress2026!Secure' );
define( 'DB_HOST', 'localhost' );
```

5.4 Configuration des permissions

```
# Propriétaire correct (Apache)
chown -R www-data:www-data /var/www/wordpress

# Permissions sur les dossiers
chmod -R 755 /var/www/wordpress

# Protection wp-config.php
chmod 600 /var/www/wordpress/wp-config.php
```

5.5 Suppression des fichiers inutiles

```
# Supprimer fichiers de documentation
rm /var/www/wordpress/readme.html
rm /var/www/wordpress/license.txt

# Nettoyer /tmp
```



```
cd ~  
rm -rf /tmp/wordpress  
rm /tmp/latest.tar.gz
```

5.6 Lancement de l'installation via l'interface web

Accès : `http://172.16.114.136/` ou `http://wordpress.local/`

Informations fournies lors de l'installation :

- **Titre du site :** Projet Cyber- Thanina
- **Nom d'utilisateur :** [Thanina_admin]
- **Mot de passe :** [RTYfghvbn16]
- **E-mail :** [thanina.1607@gmail.com]
- **Visibilité :** Décoché

PARTIE 1 - Section 6 : Accès au service

6.1 Configuration DNS local (fichier hosts)

Sur la machine Windows :

1. Ouvrir le Bloc-notes en administrateur
2. Ouvrir : `C:\Windows\System32\drivers\etc\hosts`
3. Ajouter à la fin :

```
172.16.114.136    wordpress.local
```

4. Sauvegarder le fichier

6.3 Vérification de l'accès HTTP

Dans le navigateur :

```
http://172.16.114.136
```

Résultat attendu : Page d'accueil WordPress fonctionnelle

6.4 Test du tableau de bord

Accès administration :

<http://172.16.114.136/wp-admin>

Vérifications :

- ☒ Connexion possible avec les identifiants créés
- ☒ Tableau de bord accessible
- ☒ Création de mon premier article

PARTIE 2 - Section A : Sécurisation du système Linux

A.1 Désactivation de la connexion SSH du compte root

```
# Modifier la configuration SSH
sed -i '37s/.*/PermitRootLogin no/' /etc/ssh/sshd_config

# Vérifier la modification
grep "PermitRootLogin" /etc/ssh/sshd_config
# Résultat : PermitRootLogin no
```

A.2 Mise en place de l'authentification SSH par clé

Déploiement de la clé publique

Pour l'utilisateur thanina :

```
# Passer à thanina
su - thanina

# Créer le répertoire .ssh
mkdir -p ~/.ssh
chmod 700 ~/.ssh

# Ajouter la clé publique
nano ~/.ssh/authorized_keys
# Coller la clé publique prise de mon ordi

# Permissions correctes
chmod 600 ~/.ssh/authorized_keys

# Retour root
exit
```

Pour l'utilisateur youenn :

```
su - youenn
mkdir -p ~/.ssh
chmod 700 ~/.ssh
nano ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
exit
```

A.3 Désactivation de l'authentification par mot de passe SSH

```
# Modifier la configuration SSH
sed -i '61s/./PasswordAuthentication no/' /etc/ssh/sshd_config

# Vérifier
grep "PasswordAuthentication" /etc/ssh/sshd_config
# Résultat : PasswordAuthentication no

# Redémarrer SSH
systemctl restart sshd
```

A.4 Test de connexion SSH par clé

Depuis Windows PowerShell :

```
ssh thanina@172.16.114.136
```

Résultat attendu : Connexion sans demande de mot de passe 

```
thanina@Tkana--b1-cyber: ~  
Windows PowerShell  
Copyright (C) Microsoft Corporation. Tous droits réservés.  
  
Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindows  
S  
  
PS C:\Users\thani> ssh thanina@172.16.114.136  
Linux Tkana--b1-cyber 6.8.12-16-pve #1 SMP PREEMPT_DYNAMIC PMX 6.8.12-16 (2025-10-14T08:58Z) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Sun Feb 15 16:48:00 2026 from 172.16.254.101  
thanina@Tkana--b1-cyber:~$ |
```

A.5 Suppression des comptes inutiles

```
# Lister les utilisateurs avec shell  
cat /etc/passwd | grep -E "/bin/bash|/bin/sh"  
  
# Vérifier qu'il n'y a pas de comptes inutiles  
# Comptes conservés : root, thanina, Yduval, www-data
```

A.6 Activation des mises à jour de sécurité automatiques

```
# Installer le paquet  
apt install unattended-upgrades -y  
  
# Activer les mises à jour automatiques  
echo 'APT::Periodic::Update-Package-Lists "1";' > /etc/apt/apt.conf.d/20auto-upgrades  
echo 'APT::Periodic::Unattended-Upgrade "1";' >> /etc/apt/apt.conf.d/20auto-upgrades  
  
# Vérifier la configuration  
cat /etc/apt/apt.conf.d/20auto-upgrades
```

Justification : Les mises à jour de sécurité automatiques assurent que le système est toujours à jour avec les derniers correctifs sans intervention manuelle.

A.7 Vérification de la section A

```
# Vérifier SSH
grep -E "PermitRootLogin|PasswordAuthentication" /etc/ssh/sshd_config

# Vérifier les clés SSH
ls -la /home/thanina/.ssh/
ls -la /home/Yduval/.ssh/

# Vérifier les mises à jour auto
systemctl is-enabled unattended-upgrades
```

PARTIE 2 - Section B : Pare-feu et réseau

B.1 Installation d'UFW

```
apt install ufw -y
```

B.2 Configuration des règles par défaut

```
# Bloquer tout le trafic entrant par défaut
ufw default deny incoming

# Autoriser tout le trafic sortant
ufw default allow outgoing
```

B.3 Autorisation des ports nécessaires

```
# Autoriser SSH (port 22)
ufw allow 22/tcp

# Autoriser HTTP (port 80)
ufw allow 80/tcp

# Autoriser HTTPS (port 443)
ufw allow 443/tcp
```

B.4 Activation du pare-feu

```
# Activer UFW
ufw enable

# Répondre 'y' à la confirmation
```

```
# Vérifier le statut
ufw status verbose
```

Résultat :

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
```

To	Action	From
--	-----	----
22/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere
443/tcp	ALLOW IN	Anywhere

B.5 Vérification des ports ouverts

```
# Voir tous les ports en écoute
ss -tulpn | grep LISTEN
```

Ports attendus :

- 22 (SSH) - Accessible
- 80 (HTTP) - Accessible
- 443 (HTTPS) - Accessible
- 3306 (MySQL) - Localhost uniquement 

B.6 Protection réseau supplémentaire

Vérification de la configuration MySQL :

```
grep "bind-address" /etc/mysql/mariadb.conf.d/50-server.cnf
# Résultat : bind-address = 127.0.0.1
```

PARTIE 2 - Section C : Sécurisation du serveur Apache

C.1 Masquage des informations de version

```
# Modifier la configuration de sécurité
sed -i 's/^ServerTokens.*/ServerTokens Prod/' /etc/apache2/conf-
available/security.conf
sed -i 's/^ServerSignature.*/ServerSignature Off/' /etc/apache2/conf-
available/security.conf

# Vérifier
grep -E "ServerTokens|ServerSignature" /etc/apache2/conf-
available/security.conf
```

C.2 Désactivation de l'indexation des répertoires

Déjà configuré dans le VirtualHost :

```
Options -Indexes +FollowSymLinks
```

C.3 Limitation des méthodes HTTP autorisées

Modification du VirtualHost WordPress :

```
nano /etc/apache2/sites-available/wordpress.conf
```

Ajout dans la section `<Directory>` :

```
<LimitExcept GET POST HEAD>
    Require all denied
</LimitExcept>
```

C.4 Désactivation des modules inutiles

```
# Désactiver le module autoindex (listing de répertoires)
a2dismod autoindex

# Désactiver le module status (informations serveur)
a2dismod status

# Vérifier
apache2ctl -M | grep -E 'autoindex|status'
# Ne devrait rien afficher
```

C.5 Activation de la configuration et redémarrage

```
# Activer la config security
a2enconf security

# Redémarrer Apache
systemctl restart apache2

# Vérifier
systemctl status apache2
```

C.6 Vérification de la sécurisation

```
# Tester que la version n'est pas révélée
curl -I http://localhost
# Doit afficher "Server: Apache" sans numéro de version
```

PARTIE 2 - Section D : Sécurisation HTTPS

D.1 Génération du certificat SSL auto-signé

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/ssl/private/wordpress-selfsigned.key \
    -out /etc/ssl/certs/wordpress-selfsigned.crt
```

Informations fournies :

- Country Name: FR
- State: Pays de la Loire
- Locality: Nantes
- Organization: EPSI
- Organizational Unit: []
- **Common Name:** wordpress.local
- Email: []

D.2 Création du VirtualHost SSL

```
nano /etc/apache2/sites-available/wordpress-ssl.conf
```

Contenu :


```

<VirtualHost *:443>
    ServerAdmin admin@wordpress.local
    ServerName wordpress.local
    DocumentRoot /var/www/wordpress

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/wordpress-selfsigned.crt
    SSLCertificateKeyFile /etc/ssl/private/wordpress-selfsigned.key

    <Directory /var/www/wordpress>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted

        <LimitExcept GET POST HEAD>
            Require all denied
        </LimitExcept>
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/wordpress_ssl_error.log
    CustomLog ${APACHE_LOG_DIR}/wordpress_ssl_access.log combined
</VirtualHost>

```

D.3 Configuration de la redirection HTTP → HTTPS

```
nano /etc/apache2/sites-available/wordpress.conf
```

Remplacer tout le contenu par :

```

<VirtualHost *:80>
    ServerName wordpress.local
    Redirect permanent / https://wordpress.local/
</VirtualHost>

```

D.4 Activation de SSL et du site HTTPS

```

# Activer le module SSL (déjà fait en Section 2)
a2enmod ssl

# Activer le site HTTPS
a2ensite wordpress-ssl.conf

# Tester la configuration

```

```
apache2ctl configtest
# Résultat : Syntax OK

# Recharger Apache
systemctl reload apache2
```

D.5 Vérification de l'accès HTTPS

Dans le navigateur :

```
https://wordpress.local
```

Résultat :

-  Avertissement de sécurité
-  Après acceptation : WordPress accessible en HTTPS
-  Redirection automatique depuis HTTP

D.6 Vérification des ports

```
ss -tulpn | grep -E ':80|:443'
```

PARTIE 2 - Section E : Sécurisation de PHP

E.1 Localisation du fichier php.ini

```
# Version PHP
php -v
# Résultat : PHP 8.2.29

# Chemin du php.ini pour Apache
php -i | grep "Loaded Configuration File"
# Résultat : /etc/php/8.2/apache2/php.ini
```

E.2 Configuration sécurisée de PHP (via sed)

```
# Désactiver affichage des erreurs
sed -i 's/^display_errors = On/display_errors = Off/'
/etc/php/8.2/apache2/php.ini
sed -i 's/^display_startup_errors = On/display_startup_errors = Off/'
```

```

/etc/php/8.2/apache2/php.ini

# Activer les logs d'erreurs
sed -i 's/^;log_errors = On/log_errors = On/' /etc/php/8.2/apache2/php.ini
sed -i 's/^log_errors = Off/log_errors = On/' /etc/php/8.2/apache2/php.ini
sed -i 's|^;error_log = php_errors.log|error_log = /var/log/php_errors.log|' /etc/php/8.2/apache2/php.ini

# Masquer la version PHP
sed -i 's/^expose_php = On/expose_php = Off/' /etc/php/8.2/apache2/php.ini

# Désactiver fonctions dangereuses
sed -i 's/^disable_functions = ./disable_functions = exec,passthru,shell_exec,system,proc_open,popen,curl_exec,curl_multi_exec,parse_ini_file,show_source/' /etc/php/8.2/apache2/php.ini

# Limiter uploads
sed -i 's/^upload_max_filesize = ./upload_max_filesize = 10M/' /etc/php/8.2/apache2/php.ini
sed -i 's/^post_max_size = ./post_max_size = 10M/' /etc/php/8.2/apache2/php.ini

# Limiter ressources
sed -i 's/^memory_limit = ./memory_limit = 128M/' /etc/php/8.2/apache2/php.ini
sed -i 's/^max_execution_time = ./max_execution_time = 30/' /etc/php/8.2/apache2/php.ini
sed -i 's/^max_input_time = ./max_input_time = 30/' /etc/php/8.2/apache2/php.ini

```

Justifications :

Paramètre	Valeur	Justification
display_errors	Off	Ne pas révéler d'informations système aux visiteurs
log_errors	On	Enregistrer les erreurs pour diagnostic
expose_php	Off	Ne pas révéler la version de PHP
disable_functions	[liste]	Bloquer fonctions permettant l'exécution de code
upload_max_filesize	10M	Limiter la taille des uploads (attaques DoS)
memory_limit	128M	Éviter la consommation excessive de mémoire
max_execution_time	30	Limiter le temps d'exécution (scripts malveillants)

E.3 Création du fichier de log PHP

```
touch /var/log/php_errors.log
chown www-data:www-data /var/log/php_errors.log
chmod 640 /var/log/php_errors.log
```

E.4 Redémarrage d'Apache

```
systemctl restart apache2
```

E.5 Vérification de la configuration PHP

```
# Vérifier via Apache (pas CLI)
echo "<?php phpinfo(); ?>" > /var/www/wordpress/phpinfo.php
```

Accès navigateur : <https://wordpress.local/phpinfo.php>

Vérifications dans phpinfo() :

- display_errors : Off 
- expose_php : Off 
- disable_functions : exec, passthru... 
- upload_max_filesize : 10M 

```
# Supprimer phpinfo.php (sécurité)
rm /var/www/wordpress/phpinfo.php
```

PARTIE 2 - Section F : Sécurisation de la base de données

F.1 Vérification de l'écoute MySQL sur localhost uniquement

```
# Vérifier le port MySQL
ss -tulpn | grep 3306
# Résultat : tcp LISTEN 0 80 127.0.0.1:3306 0.0.0.0:*
```

Vérification de la configuration :

```
grep "bind-address" /etc/mysql/mariadb.conf.d/50-server.cnf
# Résultat : bind-address = 127.0.0.1
```

F.2 Vérification des utilisateurs MySQL

```
mysql -u root -p
```

```
-- Voir tous les utilisateurs
SELECT user, host FROM mysql.user;
```

F.3 Suppression des comptes inutiles

```
-- Supprimer les utilisateurs anonymes (s'ils existent)
DROP USER IF EXISTS '@'localhost';
DROP USER IF EXISTS '@%';

-- Vérifier qu'il n'y a pas d'autres comptes suspects
SELECT user, host FROM mysql.user;

EXIT;
```

F.4 Vérification des droits de wp_user

```
mysql -u root -p
```

```
-- Vérifier les privilèges
SHOW GRANTS FOR 'wp_user'@'localhost';
```

wp_user a accès uniquement à wordpress_db, pas aux autres bases

F.5 Vérification de la sécurité des mots de passe

```
-- Vérifier que root ne peut se connecter que depuis localhost
SELECT user, host FROM mysql.user WHERE user='root';
-- Résultat : root | localhost

EXIT;
```

F.6 Test de connexion de l'utilisateur WordPress

```
# Tester la connexion avec wp_user
mysql -u wp_user -p

SHOW DATABASES;
-- Résultat : Uniquement wordpress_db et information_schema

EXIT;
```

PARTIE 2 - Section G : Permissions et fichiers

G.1 Définition du propriétaire correct

```
# Apache s'exécute sous l'utilisateur www-data
chown -R www-data:www-data /var/www/wordpress
```

G.2 Configuration des permissions restrictives

```
# Permissions sur les dossiers (755)
find /var/www/wordpress -type d -exec chmod 755 {} \;

# Permissions sur les fichiers (644)
find /var/www/wordpress -type f -exec chmod 644 {} \;
```

G.3 Protection spéciale pour wp-config.php

```
# Propriétaire root (sécurité renforcée)
chown root:www-data /var/www/wordpress/wp-config.php

# Permissions 640 (root peut écrire, www-data peut lire)
chmod 640 /var/www/wordpress/wp-config.php

# Vérifier
ls -la /var/www/wordpress/wp-config.php
# Résultat : -rw-r----- 1 root www-data 3334 wp-config.php
```

G.4 Suppression des fichiers inutiles

```
# Supprimer fichiers de documentation
rm -f /var/www/wordpress/readme.html
rm -f /var/www/wordpress/license.txt
```

```
# Supprimer wp-config-sample.php
rm -f /var/www/wordpress/wp-config-sample.php

# Supprimer hello.php (plugin exemple)
rm -f /var/www/wordpress/wp-content/plugins/hello.php
```

G.5 Vérification des permissions

```
# Vérifier wp-config.php
ls -la /var/www/wordpress/wp-config.php

# Vérifier quelques fichiers
ls -la /var/www/wordpress/ | head -20
```

PARTIE 2 - Section H : Sécurisation applicative WordPress

H.1 Vérification des utilisateurs WordPress

Dans l'interface d'administration :

Utilisateurs → Tous les utilisateurs

- ✓ Aucun utilisateur nommé "admin"
- ✓ Compte administrateur créé avec nom personnalisé

H.2 Modification de wp-config.php pour la sécurité

```
nano /var/www/wordpress/wp-config.php
```

Ajout avant `/* C'est tout, ne touchez pas à ce qui suit ! */ :`

```
// Sécurité WordPress
define('DISALLOW_FILE_EDIT', true);
define('FORCE_SSL_ADMIN', true);
```

Justifications :

- `DISALLOW_FILE_EDIT` : Désactive l'éditeur de thèmes/plugins dans l'interface admin, empêchant l'injection de code via l'interface
- `FORCE_SSL_ADMIN` : Force HTTPS pour toutes les pages d'administration

H.3 Installation du plugin de limitation de connexions

Dans WordPress :

Extensions → Ajouter une extension

1. Rechercher : **"Limit Login Attempts Reloaded"**
2. **Installer**
3. **Activer**

Configuration du plugin :

Réglages → Limit Login Attempts

- **Tentatives autorisées** : 3
- **Durée de verrouillage** : 20 minutes
- **Sauvegarder**

H.4 Suppression des thèmes inutilisés

Dans WordPress :

Apparence → Thèmes

- **Conserver** : Un seul thème actif
- **Supprimer** : Tous les autres thèmes

H.5 Suppression des plugins inutilisés

Dans WordPress :

Extensions → Extensions installées

- **Supprimer** : Tous les plugins désactivés non utilisés

H.6 Vérification des mots de passe

Dans WordPress :

Utilisateurs → Profil

- ✓ **Mot de passe fort utilisé**
- ✓ **Pas de mot de passe faible ou par défaut**



Section I : Journalisation et détection

I.1 Identification des fichiers de logs

```
# Logs Apache
ls -lh /var/log/apache2/wordpress_error.log
ls -lh /var/log/apache2/wordpress_access.log
ls -lh /var/log/apache2/wordpress_ssl_error.log
ls -lh /var/log/apache2/wordpress_ssl_access.log

# Logs MySQL
ls -lh /var/log/mysql/error.log

# Logs PHP
ls -lh /var/log/php_errors.log

# Logs système (via journalctl pour LXC)
journalctl -u apache2 --since today
journalctl -u mariadb --since today
journalctl -u sshd --since today
```

Note : Dans un conteneur LXC, les logs SSH sont gérés par systemd/journalctl plutôt que /var/log/auth.log.

I.2 Analyse des logs Apache

```
# Voir les dernières erreurs
tail -20 /var/log/apache2/wordpress_error.log

# Voir les derniers accès
tail -20 /var/log/apache2/wordpress_access.log

# Rechercher les erreurs 404
grep "404" /var/log/apache2/wordpress_access.log | tail -10

# Rechercher les erreurs 500
grep "500" /var/log/apache2/wordpress_error.log | tail -10
```

I.3 Analyse des logs SSH

```
# Voir les tentatives de connexion échouées
journalctl -u sshd | grep "Failed" | tail -20

# Voir les connexions réussies
journalctl -u sshd | grep "Accepted" | tail -20
```

I.4 Installation de fail2ban

```
apt install fail2ban -y
```

I.5 Configuration de fail2ban (adapté LXC)

```
# Arrêter fail2ban
systemctl stop fail2ban

# Créer la configuration
cat > /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 3
backend = systemd

[sshd]
enabled = true
port = 22
maxretry = 3
EOF

# Démarrer fail2ban
systemctl start fail2ban
systemctl enable fail2ban
```

Configuration :

- `bantime = 3600` : 1 heure de bannissement
- `findtime = 600` : 10 minutes de fenêtre de détection
- `maxretry = 3` : 3 tentatives maximum
- `backend = systemd` : Utilise journalctl (adapté pour LXC)

I.6 Vérification de fail2ban

```
# Statut général
fail2ban-client status

# Statut de la jail SSH
fail2ban-client status sshd
```

Résultat :

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed: 0
|  `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`-- Actions
    |- Currently banned: 0
    |- Total banned: 0
    `-- Banned IP list:
```

Section J : Sauvegardes

J.1 Création du répertoire de sauvegardes

```
mkdir -p /root/backups
chmod 700 /root/backups
```

J.2 Création du script de sauvegarde

```
cat > /root/backup-wordpress.sh << 'EOF'
#!/bin/bash
# Script de sauvegarde WordPress
DATE=$(date +%Y%m%d-%H%M%S)
BACKUP_DIR="/root/backups"
WP_DIR="/var/www/wordpress"
DB_NAME="wordpress_db"
DB_USER="wp_user"
DB_PASS="WordPress2026!Secure"

# Créer le répertoire de sauvegarde du jour
mkdir -p ${BACKUP_DIR}/${DATE}

# Sauvegarde des fichiers WordPress
echo "Sauvegarde des fichiers WordPress..."
tar -czf ${BACKUP_DIR}/${DATE}/wordpress-files.tar.gz ${WP_DIR}

# Sauvegarde de la base de données
echo "Sauvegarde de la base de données..."
mysqldump -u ${DB_USER} -p${DB_PASS} ${DB_NAME} >
${BACKUP_DIR}/${DATE}/wordpress-db.sql

# Compression de la sauvegarde DB
```

```

gzip ${BACKUP_DIR}/${DATE}/wordpress-db.sql

# Afficher la taille
du -sh ${BACKUP_DIR}/${DATE}

# Supprimer les sauvegardes de plus de 7 jours
find ${BACKUP_DIR} -type d -mtime +7 -exec rm -rf {} \; 2>/dev/null

echo "Sauvegarde terminée : ${BACKUP_DIR}/${DATE}"
EOF

# Rendre le script exécutable
chmod +x /root/backup-wordpress.sh

```

Composants sauvegardés :

- **Fichiers WordPress** : Tous les fichiers (code, thèmes, plugins, uploads)
- **Base de données** : Dump SQL complet de wordpress_db
- **Compression** : Réduction de l'espace disque utilisé
- **Rotation** : Suppression automatique après 7 jours

J.3 Test de la sauvegarde

```

# Exécuter manuellement
/root/backup-wordpress.sh

# Vérifier la création
ls -lh /root/backups/

```

Résultat :

```
drwxr-xr-x 2 root root 4096 Feb 15 16:02 20260215-160240
```

Contenu de la sauvegarde :

```

ls -lh /root/backups/20260215-160240/
# wordpress-files.tar.gz
# wordpress-db.sql.gz

```

J.4 Automatisation avec cron

```
# Ajouter une tâche cron (tous les jours à 2h du matin)
(crontab -l 2>/dev/null; echo "0 2 * * * /root/backup-wordpress.sh >>
/var/log/wordpress-backup.log 2>&1") | crontab -

# Vérifier la crontab
crontab -l
```

Configuration cron :

- **Fréquence** : Tous les jours à 2h00
- **Logs** : Redirection vers `/var/log/wordpress-backup.log`

Justification : Sauvegarde automatique quotidienne garantit la disponibilité des données en cas de problème.

J.5 Procédure de restauration

Restauration des fichiers

```
# Lister les sauvegardes disponibles
ls -l /root/backups/

# Restaurer les fichiers
tar -xzf /root/backups/YYYYMMDD-HHMMSS/wordpress-files.tar.gz -C /
```

Restauration de la base de données

```
# Décompresser le dump SQL
gunzip /root/backups/YYYYMMDD-HHMMSS/wordpress-db.sql.gz

# Restaurer dans MySQL
mysql -u wp_user -p wordpress_db < /root/backups/YYYYMMDD-HHMMSS/wordpress-
db.sql
```

Accès

Serveur :

- **IP** : 172.16.114.136
- **Hostname** : Tkana--b1-cyber

Accès SSH :

- **Utilisateur** : Youenn (professeur)
- **Méthode** : Clé SSH
- **Port** : 22

Accès WordPress :

- **URL** : <https://172.16.114.136>
- **Admin** : [Thanina_admin]

Accès MySQL :

- **Root** : `mysql -u root -p` (localhost uniquement)
- **Base** : `wordpress_db`
- **Utilisateur** : `wp_user`

J'ai beaucoup utilisée `Sed -i` au lieu de modifier simplement dans nano car je n'arrivais pas à faire `ctrl_W` pour trouver les lignes que je voulais donc j'ai trouvé cette commande qui est vachement utile